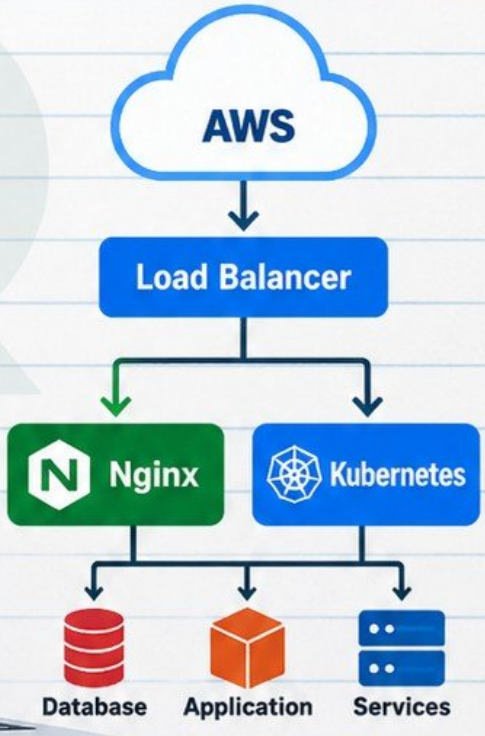
VERIQTA

The First 30 Minutes of a Production Incident

20 Real Investigation Scenarios

Introduces production engineering judgment without requiring senior-level experience





REAL INCIDENT SCENARIOS

PRACTICAL DIAGNOSTICS

ENGINEERING JUDGMENT

PRODUCTION FOCUS

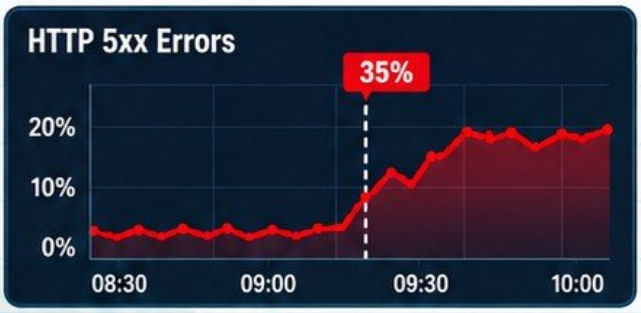
@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

The Alert Fires: What Do You Investigate First?

Scenario: At 09:17, monitoring reports a 35% increase in HTTP 5xx errors. CPU and memory appear normal.

1. Understand the Situation

- Check the alert details (time, severity, service, region).
- Confirm if customers are affected.
- Identify which endpoints or features are failing.
- Check if the issue is recent or ongoing.
- Look at related alerts (latency, 4xx/5xx, database, dependencies).



2. Check Customer Impact

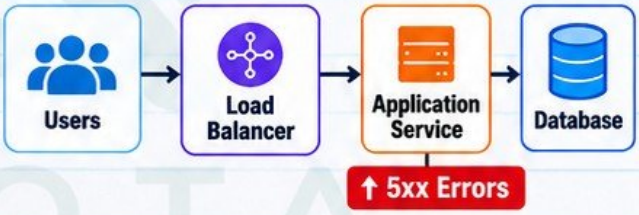
- Is the entire application affected or only specific features?
- Are real users reporting issues?
- Check error rate, request volume and latency.
- Compare with normal baseline traffic.

Check Key Metrics

Error Rate (5xx)	Request Volume	Latency (p95)
35% ↑	Normal	420 ms ↑

3. Identify the Affected Service

- Use monitoring and logs to find which service is returning 5xx.
- Check if it's the application, API, database or a dependency.
- Look at service dashboards, logs and traces.



4. Check Recent Changes

- Was there a recent deployment?
- Any infrastructure change (scaling, config, new version)?
- Check change logs, CI/CD pipeline, and infrastructure events.

Recent Events (Timeline)

09:05	Deployment completed (v1.4.3)
09:12	New pods started
09:17	HTTP 5xx errors increased
09:20	On-call engineer notified

5. Start an Incident Timeline

- Note the time the issue started.
- Record key events, observations and actions.
- Include what you checked and what you found.
- This helps with communication and later root cause analysis.

Initial Incident Note (Example)

09:17	Alert: 5xx error rate at 35%
09:19	Checked metrics: CPU & memory normal
09:21	Confirmed customer reports
09:24	Investigating application logs

Why You Should NOT Restart the Service Immediately

- You may lose important logs and evidence.
- You might hide the real cause.
- The issue could be in a dependency, not the service itself.
- A restart can make recovery slower if the real problem is not fixed.

Next Investigation Steps

1. Check application and access logs.
2. Compare with baseline traffic and metrics.
3. Verify recent deployments and configuration changes.
4. Investigate dependencies (database, external APIs, etc.).
5. Communicate your findings to the team.

The Service Is Active, but Customers Cannot Connect

Scenario: systemctl status shows active (running), but customers receive connection errors.

1. Check Service Status

$ systemctl status myapp

```
● myapp.service - My Application
   Loaded: loaded (/etc/systemd/system/myapp.service)
   Active: active (running) since Tue 2026-10-28 09:10:12
 Main PID: 1234 (myapp)
    Tasks: 20 (limit: 4915)
   Memory: 200.5M
   CGroup: /system.slice/myapp.service
           └─1234 /usr/bin/myapp
```

This shows the process is running.
It does not confirm that the application is reachable or working correctly.

2. Check If the Application Is Listening

$ ss -lntp | grep :8080

```
LISTEN   0   128   0.0.0.0:8080   0.0.0.0:*
users:(("myapp",pid=1234,fd=7))
```

Application is listening on all interfaces

$ ss -lntp | grep :8080

```
LISTEN   0   128   127.0.0.1:8080   0.0.0.0:*
users:(("myapp",pid=1234,fd=7))
```

Application is only listening on localhost (not accessible from external clients)

3. Test the Application Locally

$ curl -v http://127.0.0.1:8080

```
*   Trying 127.0.0.1:8080...
* Connected to 127.0.0.1 (127.0.0.1) port 8080
> GET / HTTP/1.1
> Host: 127.0.0.1:8080
> User-Agent: curl/8.0
>
< HTTP/1.1 200 OK
< Content-Type: application/json
{ "status": "ok" }
```

If this works, the application is responding locally.
If it fails, check logs and configuration.

4. Test External or Public Access

$ curl -v http://<server-ip>:8080

```
*   Trying 34.120.10.5:8080...
* connect to 34.120.10.5 port 8080 failed:
  Connection refused
* Failed to connect to 34.120.10.5 port 8080 after
  2 ms: Could not connect to server
```

Connection refused means the application is not listening on the expected interface or a firewall is blocking the port.

5. Check Application Logs

$ journalctl -u myapp -n 20

```
Oct 28 09:15:10 server myapp[1234]: Starting server...
Oct 28 09:15:11 server myapp[1234]: Listening on 127.0.0.1:8080
Oct 28 09:16:02 server myapp[1234]: Error: failed to bind 0.0.0.0
Oct 28 09:16:02 server myapp[1234]: Permission denied
```

Logs can show binding errors, permission issues, configuration problems, or application crashes.

6. Understand the Flow



Key Judgment

A running process is not proof of a working application. Always verify that the application is listening, responding locally, and reachable from the expected network.

The CPU Is at 98%, but What Is Actually Consuming It?

Scenario: Application latency rises while one Linux server reports sustained CPU saturation.

1. Understand the Key Terms

CPU Utilization	Percentage of CPU being used. 98% means the CPU is almost fully busy.
Load Average	Average number of runnable tasks waiting for CPU (not a percentage).
Runnable Tasks	Processes ready to run but waiting for CPU time.
CPU Pressure	Shows if the system is under sustained CPU stress over time.

2. Check Overall CPU and Load

$ uptime

```
09:21:15 up 10 days,  3:12,  2 users,  load average: 8.45, 7.92, 6.31
```

High load average (more runnable tasks than CPU cores)

Indicates CPU is under heavy load over time

$ top -bn1 | head -5

```
top - 09:21:20 up 10 days,  3:12,  2 users,  load average: 8.45, 7.92, 6.31
Tasks: 235 total,   8 running, 227 sleeping,   0 stopped,   0 zombie
%Cpu(s): 98.7 us,  0.5 sy,  0.0 ni,  0.3 id,  0.1 wa,  0.4 hi,  0.0 si,  0.0 st
```

CPU utilization is 98.7% (almost fully used)

3. Identify the Process Consuming CPU

$ ps -eo pid,comm,%cpu --sort=-%cpu | head

PID	COMMAND	%CPU
1234	python	412.5
5678	java	35.1
2345	postgres	12.3
7890	nginx	5.4
1122	node	4.1
3344	containerd	2.8
9988	sshd	0.6
1167	systemd	0.3
2211	kworker	0.2
4433	rsyslogd	0.1

This process is using the most CPU. Now investigate what it is doing.

4. Investigate the Process Further

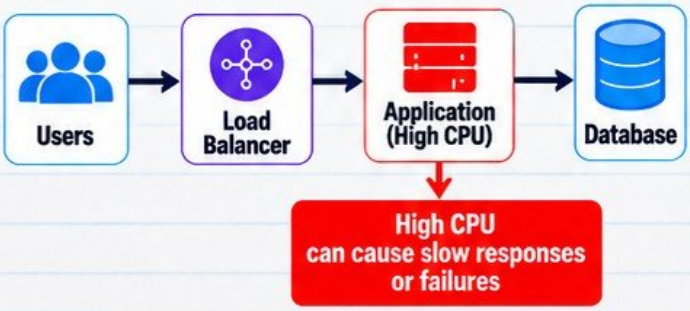
$ ps -p 1234 -o pid,ppid,comm,%cpu,%mem --forest

PID	PPID	COMMAND	%CPU	%MEM
1234	1	python	412.5	10.2
├1235	1234	python	398.1	10.0
└1236	1234	python	5.4	0.2

- Check what the process is doing (application, batch job, loop, etc).
- Look at its logs and command line.
- Determine if it is expected behavior or a runaway process.
- Check if it is directly impacting customer-facing requests.

5. Check if It Is Causing Customer Impact

- Compare the time CPU increased with increased latency/errors.
- Check application metrics (response time, error rate, request volume).
- Verify if the service using CPU is part of the customer-facing path.
- Look at logs and traces to confirm impact.



6. Common Mistake

Killing the busiest process without identifying its role.

- It might be a critical application process.
- It might be a temporary spike.
- It might restart automatically.
- You could make the outage worse.

7. Next Steps

1. Identify the process and understand what it does.
2. Check application logs and recent changes.
3. Verify if it is expected or abnormal behavior.
4. If it is a customer-impacting process, follow the proper incident procedure before taking action.

The Server Has Memory Available, but the Application Keeps Dying

Scenario: A Linux application disappears unexpectedly and restarts repeatedly.

1. Check Memory on the Server

$ free -h

```
               total        used        free      shared  buff/cache   available
Mem:            15G        6.2G        5.8G        512M        3.0G        8.1G
Swap:          2.0G        512M        1.5G
```

This shows the server has available memory.
However, the application can still be killed if it exceeds its own memory limit (systemd, container, or cgroup).

2. Check If the Application Is Restarting

$ systemctl status myapp

```
● myapp.service - My Application
     Loaded: loaded (/etc/systemd/system/myapp.service)
     Active: active (running)
   Main PID: 2481 (myapp)
      Tasks: 12 (limit: 4915)
     Memory: 1.9G

Oct 28 09:14:21 server systemd[1]: myapp.service:
        Main process exited, code=killed, status=9/KILL
Oct 28 09:14:21 server systemd[1]: myapp.service: Failed
Oct 28 09:14:21 server systemd[1]: myapp.service: Scheduled restart job
```

The application is being killed and restarted repeatedly.

3. Check Kernel Logs for Out-of-Memory Killer

$ journalctl -k | grep -i oom

```
Oct 28 09:14:20 server kernel: Out of memory: Killed
      process 2481 (myapp) total-vm:2147488kB,
      anon-rss:1801236kB, file-rss:0kB, shmem-rss:0kB,
      UID:1000 pgtables:4160kB oom_score_adj:0
Oct 28 09:14:20 server kernel: Killed process 2481
      (myapp) score 982 or sacrifice child
```

The Linux kernel killed the application because it used more memory than allowed (OOM kill).

4. Check Application Memory Limits (Systemd)

$ systemctl show myapp | grep -E "Memory"

```
MemoryCurrent=1.9G
MemoryMax=2G
MemoryHigh=1.5G
MemorySwapMax=1G
```

The service is limited to 2G of memory

Even though the server has 5.8G free, this service is limited to 2G. If it uses more, it will be killed.

5. Check Container Memory Limits (If Using Docker/Kubernetes)

$ docker inspect myapp | grep -i memory

```
        "Memory": 2147483648,
        "MemorySwap": 2147483648,
```

This container is limited to 2G of memory.
If it uses more, the container will be killed.

6. Identify the Root Cause

- Application memory leak
- Large data processing or high traffic
- Insufficient memory limit for the workload
- Unexpected memory spike after a deployment
- Inefficient code or resource usage

7. Common Mistake

- Restarting the service repeatedly without checking logs.
- Increasing memory limits without understanding the cause.
- Ignoring OOM kill messages.
- Assuming available server memory means the application can use it all.

0. Key Judgment

Available host memory does not guarantee that a memory-limited service can allocate more.

Always check application or container memory limits and investigate OOM kill logs before making changes.

The Disk Is Full, but Deleting Files Does Not Free Space

Scenario: A production application cannot write logs, even after an administrator deletes a large log file.

1. Check Disk Usage

$ df -h

```
Filesystem      Size  Used Avail Use% Mounted on
/dev/xvda1       50G   49G   1G   98% /
tmpfs           2.0G   12M  1.9G   1% /dev/shm
/dev/xvdb       200G  120G   80G   60% /data
```

The root filesystem / is 98% full.
Applications may not be able to write new files.

2. Check Inode Usage (File Count)

$ df -i

```
Filesystem      Inodes   IUsed    IFree  IUse% Mounted on
/dev/xvda1     3276800 3265000    11800   99% /
tmpfs           512000     400   511600    1% /dev/shm
/dev/xvdb     13107200 1002000 12105200    8% /data
```

Inode usage is 99%.
Even if you delete files, you cannot create new files because there are no free inodes.

3. Find Large Directories

$ du -xhd1 / | sort -h

```
4.0K    /boot
120M    /home
2.1G    /var
8.5G    /usr
12G     /opt
26G     /var/log
49G     /
```

/var/log is using 26G.
This helps identify where the space is being used.

4. Check for Deleted Files Still Open

$ lsof +L1

```
COMMAND  PID  USER    FD  TYPE  SIZE  NAME
nginx    1234 www-data 7u  REG   2.1G  /var/log/nginx/access.log (deleted)
java     5678 app      8u  REG   5.3G  /opt/app/app.log (deleted)
python   9012 app      3u  REG   1.8G  /var/log/app.log (deleted)
```

These files were deleted but are still open by running processes, so the disk space is not released.

5. Understand Why Deleting Files May Not Free Space

1. The file is deleted but still open by a running process.
2. The filesystem has no free inodes (file count is exhausted).
3. The file is on a different filesystem.
4. The process keeps writing to the file.
5. The deleted file will only be removed when the process closes it.

6. How to Free the Space Safely

- Identify which process is holding the deleted file.
- Rotate or restart the application log safely.
- Use logrotate instead of manual deletion.
- Archive or move old logs to another disk.
- Clean unnecessary files, but verify before deleting.

7. Common Mistakes

- Deleting files blindly in critical directories.
- Ignoring inode exhaustion.
- Killing a process without understanding its role.
- Assuming df -h low free space always means a large file.
- Deleting important application or database files.

8. Production Lesson

Never delete production data blindly to recover disk space.

- Always identify what is using the space.
- Check for open deleted files and inode usage.
- Use proper log rotation and retention policies.
- Follow change management in production.

The Application Works Locally, but Fails Through Nginx

Scenario: curl http://127.0.0.1:3000 succeeds, while customers receive HTTP 502 errors.



1. Verify the Application Works Locally

$ curl -v http://127.0.0.1:3000

```
*   Trying 127.0.0.1:3000...
* Connected to 127.0.0.1 (127.0.0.1) port 3000
> GET / HTTP/1.1
> Host: 127.0.0.1:3000
< HTTP/1.1 200 OK
< Content-Type: application/json
{ "status": "ok" }
```

The application is running and responding locally on port 3000.

2. Check Nginx Configuration

$ nginx -t

```
nginx: the configuration file /etc/nginx/nginx.conf
syntax is ok
nginx: configuration file /etc/nginx/nginx.conf
test is successful
```

Nginx configuration is valid.

Check Upstream Configuration

```
upstream myapp {
    server 127.0.0.1:3000;
}

server {
    listen 80;
    server_name example.com;
    location / {
        proxy_pass http://myapp;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Verify the upstream address and port match your application.

3. Check if the Application Is Listening

$ ss -lntp | grep :3000

```
LISTEN   0      128      127.0.0.1:3000      0.0.0.0:*
users:("node",pid=1234,fd=21)
```

The application is listening on 127.0.0.1:3000 (localhost only).

If it is only listening on 127.0.0.1, Nginx must be on the same server. External servers cannot access 127.0.0.1.

4. Check Nginx Error Logs

$ tail -n 20 /var/log/nginx/error.log

```
2026/10/28 09:17:42 [error] 12345#12345:
  connect() failed (111: Connection refused) while
  connecting to upstream, client: 203.0.113.10,
  server: example.com, request: "GET / HTTP/1.1",
  upstream: "http://127.0.0.1:3000/"
```

Nginx cannot connect to the upstream application. This results in HTTP 502 errors for customers.

5. Test the Nginx to Upstream Connection

$ curl -v http://127.0.0.1:3000

```
*   Trying 127.0.0.1:3000...
* Connected to 127.0.0.1 (127.0.0.1) port 3000
> GET / HTTP/1.1
> Host: 127.0.0.1:3000
< HTTP/1.1 200 OK
< Content-Type: application/json
```

If this succeeds, Nginx should also be able to connect unless there is a firewall, permission or network issue.

6. Common Causes

- Application not listening on the expected interface or port.
- Incorrect upstream address or port in Nginx.
- Firewall blocking the connection.
- Application crashing or overloaded.
- Nginx and application running on different servers with wrong network configuration.

7. Next Investigation Steps

1. Verify application is listening and healthy.
2. Check Nginx upstream configuration.
3. Check firewall and network connectivity.
4. Review Nginx error logs.
5. Check application logs for crashes or high errors.
6. Monitor resources (CPU, memory) on the application server.

8. Key Judgment

Find the failing connection before changing proxy configuration. A running application does not guarantee it is reachable through Nginx.

DNS Resolves, but the Website Still Does Not Load

Scenario: Users report intermittent failures. DNS queries return an address, but HTTPS requests time out.

1. **DNS Resolution** (domain → IP address)
2. **TCP Connectivity** (connect to IP and port 443)
3. **TLS Negotiation** (SSL handshake)
4. **HTTP Response** (request and receive data)

1. Check DNS Resolution

$ dig example.com

```
;; ANSWER SECTION:
example.com.        300    IN    A    93.184.216.34
example.com.        300    IN    A    93.184.216.35
```

DNS successfully resolves the domain to an IP address.
This only confirms that DNS is working.

2. Check TCP Connectivity (Port 443)

$ curl -v https://example.com

```
*   Trying 93.184.216.34:443...
*   connect to 93.184.216.34 port 443 failed:
    Connection timed out
*   Failed to connect to example.com port 443
    after 10 ms: Connection timed out
```

The server IP is reachable, but the TCP connection to port 443 is timing out. This could be a network issue, firewall, or server problem.

3. Check TLS Negotiation

$ openssl s_client -connect example.com:443

```
CONNECTED(00000003)
---
SSL handshake has read 0 bytes and written 324 bytes
connect: Connection timed out
```

TLS handshake failed.
The server did not respond.

4. Check HTTP Response (If TLS Works)

$ curl -v https://example.com

```
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.0
> Accept: */*
< HTTP/1.1 200 OK
< Content-Type: text/html
< Content-Length: 1256
```

If you receive a 200 OK response, DNS, TCP, TLS and HTTP are all working correctly.

5. Identify Where the Failure Is

Step	What to Check	Command	What It Tells You
DNS	Domain resolves	dig example.com	Returns IP address
TCP	Port 443 is reachable	curl -v https://example.com	Can connect to server
TLS	SSL handshake	openssl s_client -connect example.com:443	Certificate and handshake success
HTTP	Website response	curl -v https://example.com	Server returns content (e.g. 200 OK)

6. Common Mistake

Assuming successful DNS resolution proves that the website is reachable.

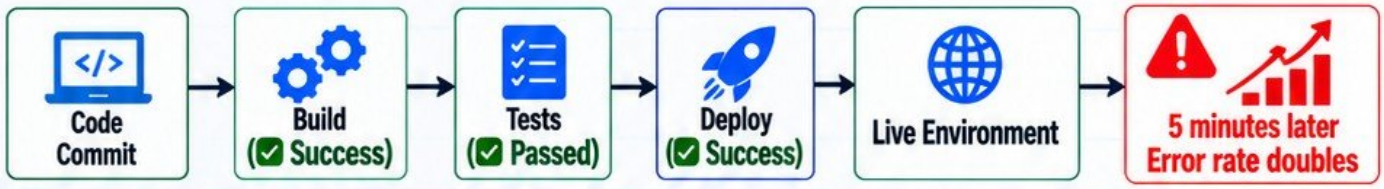
- DNS only gives you an IP address.
- The server might be down or blocking connections.
- A firewall or security group might be blocking port 443.
- TLS can fail due to certificate or protocol issues.
- The application might be returning errors.

7. Key Judgment

Verify each layer (DNS → TCP → TLS → HTTP) before changing any configuration. Do not assume the problem is with Nginx, the application, or DNS without evidence.

The Deployment Succeeded, but the Error Rate Doubled

Scenario: A CI/CD pipeline reports success. Five minutes later, customer requests begin failing.



1. Check Deployment Timestamp and Version

```
$ kubectl get deployments myapp -n production
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
myapp	3/3	3	3	2d
myapp	3/3	3	3	5m (latest)

A new deployment was rolled out 5 minutes ago.

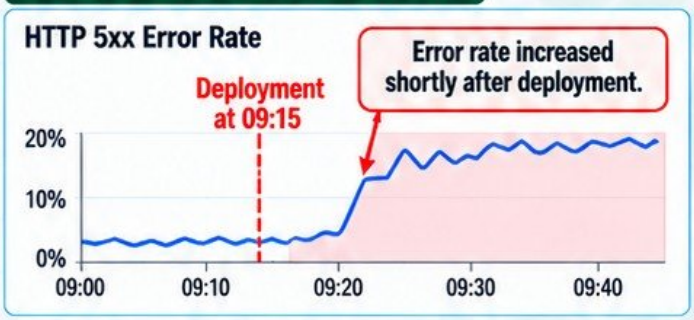
2. Compare Application Versions

```
$ kubectl get pods -n production -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	VERSION
myapp-7d5f6c9b7-abc1	1/1	Running	0	5m	v2.1.0
myapp-7d5f6c9b7-abc2	1/1	Running	0	5m	v2.1.0
myapp-6c8d9f4bb-old1	1/1	Terminated	0	12h	

New version v2.1.0 is now running in production.

3. Check Error Rate Dashboard



4. Check Application Logs

```
$ kubectl logs -n production -l app=myapp --tail=20
```

```
2026-10-28 09:17:52 ERROR Database connection failed
2026-10-28 09:17:53 ERROR Timeout while calling upstream
2026-10-28 09:17:54 ERROR NullPointerException
2026-10-28 09:17:55 ERROR HTTP 500 Internal Server Error
```

Errors started immediately after the new version was deployed.

5. Check Health Checks and Service Status

```
$ kubectl get pods,svc -n production
```

NAME	READY	STATUS	RESTARTS	AGE
pod/myapp-7d5f6c9b7-abc1	1/1	Running	0	5m
pod/myapp-7d5f6c9b7-abc2	1/1	Running	0	5m

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
svc/myapp	ClusterIP	10.96.12.45	80/TCP	2d
svc/myapp	LoadBalancer	34.118.10.20	80:32456/TCP	2d

Pods are running and healthy, but customers are still experiencing errors.

6. Investigate Further

1. Review deployment logs (CI/CD pipeline output).
2. Compare changes between v2.0.1 and v2.1.0.
3. Check configuration changes, environment variables, and database migrations.
4. Verify external dependencies (databases, APIs, caches).
5. Assess rollback readiness if the issue is related to the new deployment.

7. Successful Deployment vs Successful Operation

Successful Deployment Execution

- Code built successfully
- Tests passed
- Deployment completed without errors
- Pods/servers are running

Successful Production Operation

- Application is working correctly
- No significant errors
- Customers can access the service
- All dependencies are healthy
- Performance is normal

8. Key Judgment

A recent deployment is a strong hypothesis, not automatic proof of causation.

- Use evidence to confirm or rule it out.
- Correlate timestamps, logs, metrics, and changes.
- Avoid immediate rollback without understanding the impact.
- Make an informed decision based on data.

The Docker Container Exits Immediately After Starting

Scenario: A container starts, exits with a nonzero status, and is repeatedly restarted by its configured restart policy.

1. Check Container Status and Exit Code

$ docker ps -a

```
CONTAINER ID   IMAGE          COMMAND   STATUS                     NAMES
a1b2c3d4e5f6   myapp:latest   "/app"    Exited (1) 5 seconds ago   myapp
f6e5d4c3b2a1   myapp:latest   "/app"    Exited (137) 2 minutes ago myapp
```

The container is exiting with a nonzero status code. This means the application inside the container is failing.

2. Check Container Logs

$ docker logs myapp

```
2026-10-28 09:17:12  INFO   Starting application...
2026-10-28 09:17:13  ERROR  Failed to connect to database
2026-10-28 09:17:13  ERROR  Connection refused
2026-10-28 09:17:13  ERROR  Application exiting...
```

Application logs show the reason the container is exiting. Check for configuration errors, missing environment variables, or dependency connection failures.

3. Inspect Container Configuration

$ docker inspect myapp

```
{
  "Name": "/myapp",
  "State": {
    "Status": "exited",
    "ExitCode": 1,
    "StartedAt": "2026-10-28T09:17:12Z",
    "FinishedAt": "2026-10-28T09:17:13Z"
  },
  "RestartPolicy": {
    "Name": "always",
    "MaximumRetryCount": 0
  }
}
```

Nonzero exit code means the application failed.

The container is configured to restart automatically, so it keeps exiting and restarting.

4. Common Exit Codes

Exit Code	Meaning	Common Cause
0	Success	Normal exit
1	General error	Application error, bad configuration
127	Command not found	Incorrect startup command
137	Killed (OOM)	Out of memory (SIGKILL)
143	Terminated	Graceful shutdown (SIGTERM)

5. Check Resource Limits (Memory)

$ docker stats myapp

```
CONTAINER   CPU %   MEM USAGE / LIMIT   MEM %   NET I/O
myapp       0.00%   1.98GiB / 2GiB      99.0%   1.2MB / 0.5MB
```

The container may be killed due to memory limits (Exit code 137). Check if the application requires more memory.

6. Verify Startup Command and Environment

$ docker inspect myapp | grep -A 5 "Cmd"

```
"Cmd": [
  "node",
  "server.js"
],
"Env": [
  "NODE_ENV=production",
  "DB_HOST=database",
  "DB_PORT=5432"
],
```

Check that the startup command is correct and all required environment variables are set. A missing or incorrect configuration can cause the container to exit immediately.

7. Common Mistakes

- Missing environment variables.
- Incorrect startup command.
- Application configuration errors.
- Database or external service connections failing.
- Insufficient memory limits.
- Assuming the container works because it builds successfully.

8. Key Judgment

Rebuilding an image will not automatically fix a runtime configuration problem.

Always identify the actual reason for the container failure using logs, exit codes, and configuration details before making changes.

Kubernetes Reports CrashLoopBackOff: Where Is the Evidence?

Scenario: A newly deployed Pod repeatedly crashes while other replicas continue serving traffic.



1. Check Pod Status

```
$ kubectl get pods -n production
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-7d5f6c9b7-abc1	1/1	Running	0	5m
myapp-7d5f6c9b7-abc2	1/1	Running	0	5m
myapp-7d5f6c9b7-abc3	0/1	CrashLoopBackOff	5 (30s)	3m

The Pod is in CrashLoopBackOff and has restarted multiple times, while other replicas are healthy.

2. Describe the Pod for Details

```
$ kubectl describe pod myapp-abc3 -n production
```

```
Name:         myapp-7d5f6c9b7-abc3
Namespace:    production
Status:       CrashLoopBackOff
IP:           10.244.1.23
Containers:
  myapp:
    State:       Waiting
    Reason:      CrashLoopBackOff
    Last State:  Terminated
      Reason:    Error
      Exit Code: 137
      Started:   Mon, 28 Oct 2026 09:17:12
      Finished:  Mon, 28 Oct 2026 09:17:15
```

The container is repeatedly crashing with exit code 137 (killed).

3. Check Current and Previous Container Logs

```
$ kubectl logs myapp-abc3 -n production
2026-10-28 09:17:12 Starting application...
2026-10-28 09:17:14 Connecting to database...
2026-10-28 09:17:15 fatal: could not connect to database
2026-10-28 09:17:15 exiting with code 1
```

```
$ kubectl logs myapp-abc3 -n production --previous
2026-10-28 09:16:52 Starting application...
2026-10-28 09:16:54 Error: missing environment variable
2026-10-28 09:16:54 exiting with code 1
```

Previous logs show the reason the container exited. This helps you identify the real cause.

4. Check Kubernetes Events

```
$ kubectl get events -n production --sort-by='.lastTimestamp'
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
3m	Normal	Scheduled	pod/myapp-abc3	Successfully assigned to node-1
3m	Normal	Pulled	pod/myapp-abc3	Container image pulled
3m	Normal	Created	pod/myapp-abc3	Created container
3m	Normal	Started	pod/myapp-abc3	Started container
3m	Warning	BackOff	pod/myapp-abc3	Back-off restarting failed container

Events show the container is starting and then failing, which triggers the restart back-off.

5. Common Causes

- Application crashes due to code or configuration errors.
- Missing or incorrect environment variables.
- Cannot connect to a database or external service.
- Insufficient memory or CPU (OOMKilled).
- Incorrect startup command or arguments.
- Application fails health checks and exits.
- Permissions or file access issues.

6. Investigation Steps

1. Check Pod status and restart count.
2. Describe the Pod for detailed state and events.
3. Review current and previous container logs.
4. Identify exit code and reason for termination.
5. Check configuration, environment variables, and secrets.
6. Verify connectivity to dependent services.
7. Fix the root cause and redeploy.

7. Common Mistake

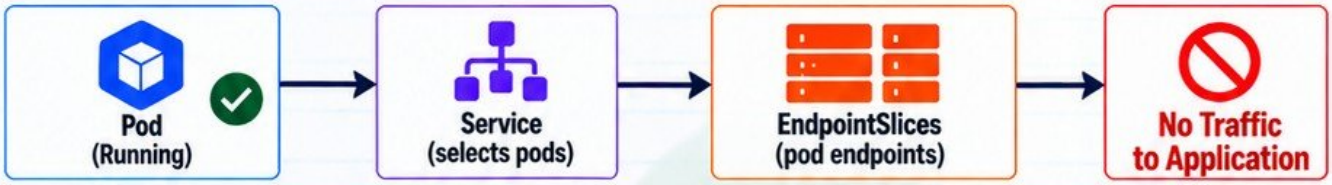
Deleting the Pod repeatedly instead of diagnosing why the container exits. This hides the evidence and delays the real fix.

8. Key Judgment

CrashLoopBackOff is a symptom, not the root cause. Find and fix the reason the container exits, using logs, events, exit codes, and configuration details.

The Kubernetes Pod Is Running, but Receives No Traffic

Scenario: Pods show Running, but users cannot reach the application through its Service.



1. Check Pod Status and Readiness

$ kubectl get pods -n production -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
myapp-7d5f6c9b7-abc1	1/1	Running	0	5m	10.244.1.12	node-1
myapp-7d5f6c9b7-abc2	1/1	Running	0	5m	10.244.2.8	node-2
myapp-7d5f6c9b7-abc3	1/1	Running	0	5m	10.244.3.15	node-3

Pods are running and ready, but this does not guarantee they are receiving traffic.

2. Check Service Configuration

$ kubectl get svc -n production

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
myapp	ClusterIP	10.96.12.45	<none>	80/TCP	5m
myapp-api	NodePort	10.96.83.21	<none>	8080:32080/TCP	5m
myapp-lb	LoadBalancer	10.96.44.11	34.118.10.20	80:443/TCP	5m

Verify the Service type, port and targetPort.
A mismatch can cause the Service to receive traffic but not forward it to the pods.

3. Check EndpointSlices (Pod Endpoints)

$ kubectl get endpointslices -n production

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
myapp-abc1	IPv4	80	10.244.1.12	5m
myapp-abc2	IPv4	80	10.244.2.8	5m
myapp-abc3	IPv4	80	10.244.3.15	5m

EndpointSlices should list the pod IPs and ports.
If there are no endpoints, the Service has no healthy pods to send traffic to.

4. Describe the Service

$ kubectl describe svc myapp -n production

```
Name:              myapp
Namespace:         production
Selector:          app=myapp
Type:              ClusterIP
IP:                10.96.12.45
Port:              80/TCP
TargetPort:        8080/TCP
Endpoints:         10.244.1.12:8080, 10.244.2.8:8080, 10.244.3.15:8080
Session Affinity:  None
```

Ensure the selector matches the pod labels and the targetPort matches the container port.

5. Verify the Application Is Listening Inside the Pod

$ kubectl exec -it myapp-abc1 -n production -- ss -lntp

State	Recv-Q	Send-Q	Local Address:Port	Process
LISTEN	0	128	0.0.0.0:8080	users:("app",pid=1)
LISTEN	0	128	[::]:8080	users:("app",pid=1)

The application must be listening on the expected port (0.0.0.0 or the pod IP) inside the container.

6. Common Causes

- Service selector does not match pod labels.
- No endpoints in EndpointSlices (pods not ready).
- Incorrect targetPort or container port.
- Application not listening on the expected port.
- Readiness probe failing (pod marked not ready).
- NetworkPolicy blocking traffic.
- Using the wrong Service type or external configuration.
- DNS or load balancer pointing to the wrong Service.

7. Key Judgment

Running, Ready, and reachable represent different conditions.

- Running: the container process is running.
- Ready: the pod has passed readiness checks.
- Reachable: the Service can successfully send traffic to the pod.

All three must be true for users to access the application.

8. Investigation Steps

1. Check pod status and readiness.
2. Verify Service configuration and selectors.
3. Confirm EndpointSlices have healthy endpoints.
4. Ensure the application is listening on the correct port.
5. Check readiness probes and pod logs.
6. Validate network policies, DNS and external access if applicable.

The Load Balancer Marks Every Target Unhealthy

Scenario: An AWS Application Load Balancer returns 503 because its target group has no healthy registered targets.



1. Check Target Health Status

```
$ aws elbv2 describe-target-health --target-group-arn <tg-arn>
```

Target	Port	State	Reason
10.96.12.45	80	unhealthy	Health checks failed
10.96.12.46	80	unhealthy	Health checks failed
10.96.12.47	80	unhealthy	Health checks failed

The target group has no healthy registered targets, which causes the ALB to return 503 errors.

2. Review ALB Health Check Configuration

Health check settings (ALB Console)

Protocol	Path	Port
HTTP	/health	80

Healthy threshold	Unhealthy threshold	Timeout (seconds)	Interval (seconds)
2	2	5	30

Verify that the health check path, port, and expected response code (200-399) match your application.

3. Verify Application Response on the Target

```
$ curl -I http://<target-ip>:80/health
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15
```

The application must return a successful response (e.g. 200) on the health check path from the target.

4. Check Security Groups and Network Access



Ensure the target security group allows traffic from the ALB security group on the correct port.

5. Check Application Logs

```
$ tail -n 20 /var/log/app.log
```

```
2026-10-28 09:15:12  INFO   Starting application...
2026-10-28 09:15:18  ERROR  Failed to connect to database
2026-10-28 09:15:18  ERROR  Health check endpoint failing
2026-10-28 09:15:18  INFO   Application not ready
```

Application logs can reveal why the health check is failing (e.g. database connection, startup delay, configuration error).

6. Common Causes

- Incorrect health check path (e.g. / instead of /health).
- Application returns 404, 500, or redirects instead of 200.
- Wrong target port configuration.
- Security group blocking ALB health check traffic.
- Application not ready (still starting or failing dependencies).
- Health check timeout too short.
- Container or instance not listening on the expected port.

7. Common Mistake

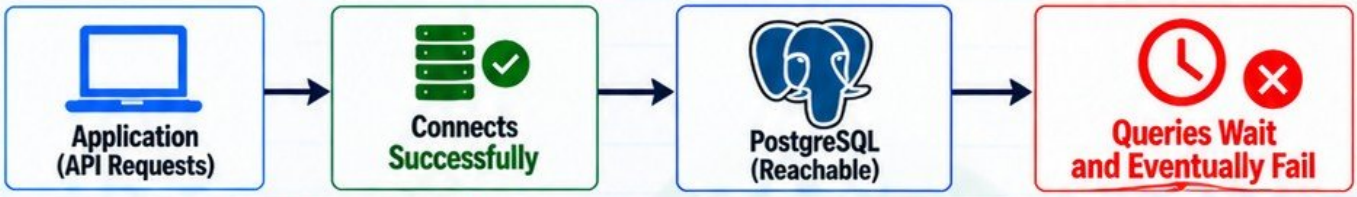
Disabling health checks or weakening security rules without identifying the real failure.
This hides the problem and can introduce security risks.

8. Key Judgment

Fix the root cause of the failed health checks.
A healthy target group requires correct health check configuration, reachable targets, and an application that returns the expected response.

The Database Is Reachable, but Requests Keep Timing Out

Scenario: The application connects to PostgreSQL, but API requests wait and eventually fail.



1. Verify Database Connectivity

```
$ psql -h <db-host> -U <user> -d <db> -c "SELECT 1;"
```

```
?column?
----------
 1
(1 row)
```

A successful connection test confirms the database is reachable, but it does not guarantee queries will complete quickly.

2. Check Connection Pool Usage (Application)

Example metrics (application or monitoring dashboard)

METRIC	VALUE	STATUS
Active connections	50	OK
Idle connections	0	LOW
Max connections	50	REACHED
Pending requests	120	HIGH
Connection pool wait time	8.5s	HIGH

When the connection pool is exhausted, new requests must wait, which causes API timeouts.

3. Check Active Queries and Long-Running Queries

```
$ sudo -u postgres psql -c "SELECT pid, now() - query_start AS duration, state, query FROM pg_stat_activity ORDER BY duration DESC LIMIT 5;"
```

PID	DURATION	STATE	QUERY
1234	00:12:35	active	SELECT * FROM orders ...
5678	00:08:21	active	UPDATE users SET ...
9012	00:05:17	active	DELETE FROM sessions ...
3456	00:02:11	idle in transaction	INSERT INTO payments ...
7890	00:01:54	active	SELECT * FROM products ...

Long-running queries can consume resources and cause other requests to wait.

4. Check for Blocked Transactions and Lock Waits

```
$ sudo -u postgres psql -c "SELECT pid, locktype, relation, mode, state, query FROM pg_locks l JOIN pg_stat_activity a ON l.pid = a.pid WHERE NOT granted;"
```

PID	LOCKTYPE	RELATION	MODE	STATE	QUERY
4321	relation	orders	ExclusiveLock	active	UPDATE orders ...
8765	relation	users	RowExclusiveLock	active	INSERT INTO users ...
2468	relation	products	ShareLock	active	SELECT * FROM products ...

Blocked transactions and lock waits can cause queries to hang, leading to API timeouts.

5. Check Database Resource Usage

```
$ sudo -u postgres psql -c "SELECT * FROM pg_stat_database WHERE datname = 'myapp';"
```

METRIC	VALUE
numbackends	50
xact_commit	125430
xact_rollback	320
blk_read_time	235600
blk_write_time	128900
temp_files	450
temp_bytes	2147483648

High number of connections, slow query times, or large temporary files can indicate database saturation.

6. Common Causes

- Connection pool exhaustion.
- Long-running queries.
- Blocked transactions and lock contention.
- Insufficient database resources (CPU, memory, disk I/O).
- Inefficient queries or missing indexes.
- Application holding connections open for too long.
- Database saturation due to high load.

7. Key Judgment

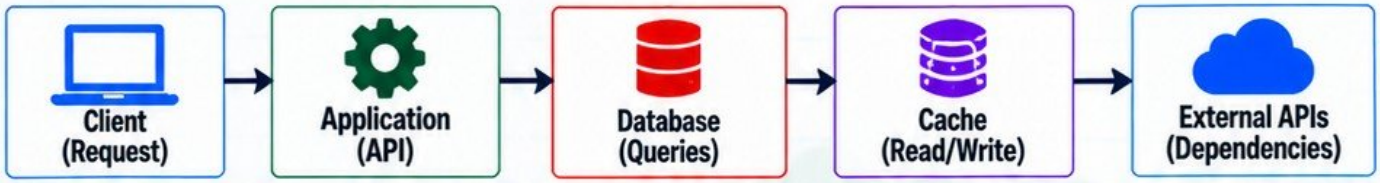
A successful database connection test does not prove queries can complete promptly. Investigate connection pool usage, query performance, locks, and database resources to find the real cause of the timeouts.

8. Investigation Steps

1. Check application connection-pool metrics.
2. Identify long-running queries.
3. Look for blocked transactions and lock waits.
4. Verify database resource usage (CPU, memory, disk, connections).
5. Optimize queries, resolve locks, or scale resources as needed.

The API Is Slow, but CPU, Memory, and Network Look Normal

Scenario: API response time rises from 180 ms to 4 seconds while infrastructure metrics remain healthy.



1. Verify Infrastructure Metrics (Confirm They Are Healthy)

$ Check CPU, memory, and network (example: Linux)

```
$ top -bn1 | head -5
%Cpu(s):   3.2 us,  1.1 sy,  0.0 ni, 95.7 id
MiB Mem :   7900.2 total,    210.5 free,
MiB Swap:   2048.0 total,   2047.1 free

$ sar -n DEV 1 5
IFACE     rxpck/s    txpck/s    rxMB/s    txMB/s
eth0      120.5      110.3      12.4      10.8
```

✓ Infrastructure metrics are normal, which means the slowdown is likely at the application or dependency level.

2. Check API Latency (p95 and p99)

Example: from monitoring dashboard or CLI

Time	p50	p95	p99
10:00	180 ms	220 ms	300 ms
10:15	750 ms	2.8 s	3.5 s
10:30	1.2 s	4.0 s	4.8 s

! p95 and p99 latency have increased significantly, confirming a real user impact.

3. Follow a Request Through Dependencies

Use distributed tracing (e.g. Jaeger, Datadog, AWS X-Ray)

Service / Operation	Duration	% of total
api-gateway	45 ms	1%
user-service	180 ms	4%
database (SELECT)	2.8 s	70%
cache (Redis GET)	120 ms	3%
external-api	800 ms	20%

The database query is the main contributor to the slow response time.

! Distributed traces show which dependency is causing the most latency.

4. Check Application and Dependency Logs

$ Search for slow requests and timeouts

```
2026-10-28 10:15:22  INFO   Request started  GET /api/orders
2026-10-28 10:15:26  WARN   Database query took 2890 ms
2026-10-28 10:15:26  WARN   External API call took 820 ms
2026-10-28 10:15:26  ERROR  Request timeout after 4000 ms
```

✓ Logs reveal slow database queries, slow external API calls, or timeouts.

5. Investigate Common Causes

- ✗ Slow database queries (missing index, full table scan).
- ✗ Cache misses or cache not working.
- ✗ Slow or failing external API calls.
- ✗ Increased downstream latency (database, third-party services).
- ✗ Synchronous calls instead of asynchronous processing.
- ✗ Application code changes causing inefficient queries or logic.
- ✗ Thread or connection pool exhaustion.
- ✗ Garbage collection or high I/O wait (not always visible in high-level metrics).

6. Investigation Steps

1. Confirm the latency increase (p95/p99).
2. Use distributed traces to identify the slow component.
3. Check database query performance and locks.
4. Verify cache hit/miss ratio and response time.
5. Check external API latency and error rates.
6. Review recent deployments or configuration changes.
7. Look for timeouts, retries, or blocking operations.

7. Key Judgment

A healthy server can still host a slow application. Normal CPU, memory, and network metrics do not guarantee that requests complete quickly.

8. Common Mistake

✗ Assuming the infrastructure is the problem and restarting instances, when the real issue is in the application or one of its dependencies.

Only One Availability Zone Is Failing

Scenario: Customers experience intermittent errors, but only requests routed to one AWS Availability Zone are affected.



1. Check Load Balancer Target Health by Zone

```
$ aws elbv2 describe-target-health --target-group-arn <tg-arn>
```

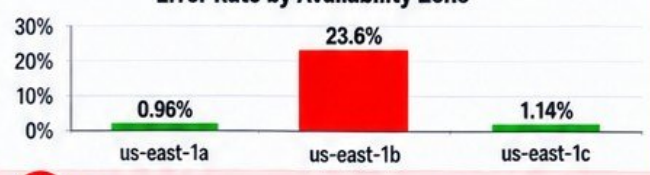
Target ID	Availability Zone	State	Reason
i-0a1b2c3d4e	us-east-1a	healthy	–
i-0b2c3d4e5f	us-east-1a	healthy	–
i-0c3d4e5f6g	us-east-1b	unhealthy	Health checks failed
i-0d4e5f6g7h	us-east-1b	unhealthy	Timeout
i-0e5f6g7h8i	us-east-1c	healthy	–
i-0f6g7h8i9j	us-east-1c	healthy	–

Targets in one Availability Zone are unhealthy, which causes intermittent errors for users routed to that zone.

2. Compare Error Rates by Availability Zone

```
$ aws cloudwatch get-metric-data --metric-data-queries ...
```

Availability Zone	Request Count	Error Count	Error Rate
us-east-1a	12,450	120	0.96%
us-east-1b	11,980	2,830	23.6%
us-east-1c	12,310	140	1.14%



Error rate is significantly higher only in us-east-1b.

3. Verify Deployment Versions Across Zones

```
$ aws ssm send-command --document-name "AWS-RunShellScript" ...
```

Instance ID	Availability Zone	Application Version
i-0a1b2c3d4e	us-east-1a	v1.4.2
i-0b2c3d4e5f	us-east-1b	v1.4.3
i-0c3d4e5f6g	us-east-1c	v1.4.2

One Availability Zone is running a different version, which may have introduced the issue.

4. Check Application and System Logs in the Failing Zone

```
$ sudo tail -n 20 /var/log/app.log
```

```
2026-10-28 09:14:32  ERROR  Database connection timeout
2026-10-28 09:14:32  WARN   Retrying request (attempt 3)
2026-10-28 09:14:33  ERROR  External API request failed
2026-10-28 09:14:34  ERROR  Upstream service unavailable
2026-10-28 09:14:35  WARN   Health check endpoint timeout
```

Logs show timeouts and dependency failures only in the failing Availability Zone.

5. Verify Network Connectivity and Dependencies

```
$ ping <db-endpoint>    $ curl -v <internal-service>
```

From Instance (AZ)	Database	Internal API	External API
us-east-1a	OK	OK	OK
us-east-1b	TIMEOUT	TIMEOUT	OK
us-east-1c	OK	OK	OK

Network or dependency access from us-east-1b is failing.

6. Common Causes

- Network issue or route table misconfiguration in one AZ.
- Security group or NACL blocking traffic in one AZ.
- Different application version or configuration.
- Dependency (database, cache, external API) unreachable from one AZ.
- Instance or container health issues in one AZ.
- Underlying AWS infrastructure issue in that AZ.

7. Key Judgment

Aggregate dashboards can conceal localized failures. Always compare metrics, target health, and logs by Availability Zone.

8. Investigation Steps

1. Check ALB target health by Availability Zone.
2. Compare error rates and request counts per AZ.
3. Verify application versions and configurations.
4. Check application logs in the failing zone.
5. Test network connectivity and dependencies.
6. Identify the root cause and remediate.

The Application Fails Only After Running for Several Hours

Scenario: A service starts normally after deployment, then gradually slows down and eventually refuses new requests.

Service Starts Normally → Runs for Several Hours → Gradually Slows Down → Eventually Refuses New Requests

1. Check Process Limits (File Descriptors)

```
$ cat /proc/<pid>/limits
```

Limit	Soft Limit	Hard Limit
Max open files	1024	4096
Max processes	32000	32000
Max memory size	unlimited	unlimited
Max stack size	8388608	unlimited

This shows the maximum number of file descriptors the process can open.

2. Check Open File Descriptors (Current Usage)

```
$ ls /proc/<pid>/fd | wc -l
```

```
9876
```

A high and growing number indicates a potential file descriptor leak.

```
$ ls /proc/<pid>/fd | head
```

```
0  /dev/null
1  /dev/pts/1
2  /dev/pts/1
3  socket:[4026532345]
4  socket:[4026532346]
5  anon_inode:[eventpoll]
7  socket:[4026532347]
8  socket:[4026532348]
9  socket:[4026532349]
9  socket:[4026532350]
```

Many open sockets or files that keep increasing over time suggest a leak.

3. Check Socket and Connection Summary

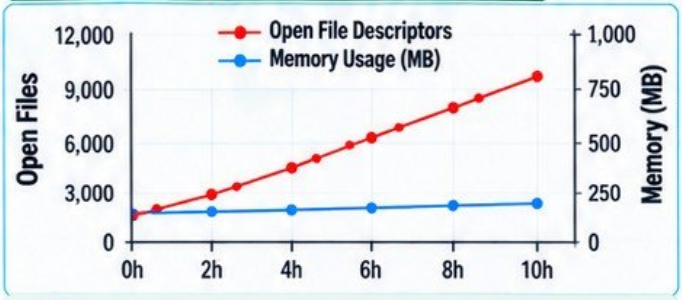
```
$ ss -s
```

```
Total: 10532 (kernel 11000)
TCP:   9840 (estab 8200, closed 1200, orphaned 0,
       timewait 440)
```

Transport	Total	IP	IPv6
*	10532	-	-
RAW	12	0	12
UDP	248	180	68
TCP	9840	6200	3640
INET	10100	6380	3720
FRAG	0	0	0

A large number of established connections or sockets can indicate connection leaks.

4. Monitor Resource Trends Over Time



A steady increase in open files, connections, or memory over time indicates a resource leak.

5. Common Causes

- File descriptor exhaustion (too many open files).
- Connection leaks (database, HTTP, or third-party services).
- Unclosed file handles or sockets.
- Long-running processes not releasing resources.
- Memory leaks leading to secondary failures.
- External service connections not being closed.
- Background jobs or threads accumulating over time.

6. Key Judgment

A restart may temporarily restore service while leaving the underlying leak unresolved.

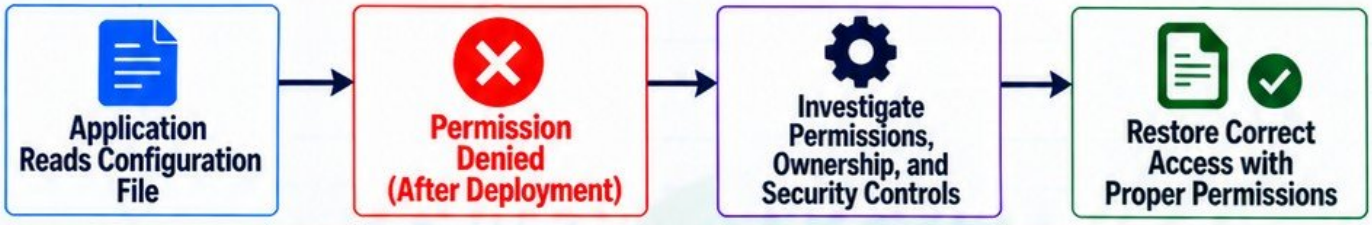
The real fix requires identifying and fixing the resource leak.

7. Investigation Steps

1. Check process limits and current usage.
2. Monitor open files, sockets, and connections over time.
3. Review application logs for repeating errors.
4. Identify components leaking resources (database, external APIs, files).
5. Reproduce the issue or analyze long-running behavior.
6. Implement a permanent fix and monitor to confirm the leak is resolved.

The Application Suddenly Returns Permission Denied

Scenario: A previously healthy application fails to read its configuration after a deployment.



1. Check the Service Identity (User and Groups)

$ id <service-user>

```
uid=995(appuser) gid=995(appuser) groups=995(appuser),
1001(logs),1002(config)
```

The application runs with a specific user and group. It can only access files and directories that permit this identity.

2. Trace Permissions Along the File Path

$ namei -l /etc/myapp/config.yaml

```
f:   mode        owner    group    name
   drwxr-xr-x    root     root     /
   drwxr-xr-x    etc      etc      etc
   drwxr-x---    root     root     myapp
   -rw-------    root     root     config.yaml
```

Each directory in the path must have execute (x) permission for the application user to traverse it.

3. Check File Ownership and Permissions

$ stat /etc/myapp/config.yaml

```
  File: /etc/myapp/config.yaml
  Size: 2453          Blocks: 8          IO Block: 4096
Access: 2026-10-28 10:15:22.000000000 +0000
Modify: 2026-10-28 09:50:10.000000000 +0000
Change: 2026-10-28 09:50:10.000000000 +0000
 Birth: -
Access: (0600/-rw-------)  Uid: (      0/root)
                           Gid: (      0/root)
```

The file is readable only by root, which causes "Permission denied" for the application user.

4. Check Application Logs for Errors

$ journalctl -u myapp -n 20 --no-pager

```
Oct 28 10:15:21 server myapp[1234]: ERROR Failed to
 read configuration file: /etc/myapp/config.yaml
Oct 28 10:15:21 server myapp[1234]: Permission denied
Oct 28 10:15:21 server myapp[1234]: Application
 startup aborted
```

Logs confirm the exact file and the reason for the failure.

5. Common Causes

- File or directory ownership changed during deployment.
- Missing read or execute permission for the application user.
- Incorrect permissions on mounted volumes (e.g. Kubernetes volumes or AWS EFS).
- Security controls: SELinux, AppArmor, or filesystem ACLs.
- Configuration file deployed with restrictive permissions (e.g. 600).
- Running the application with the wrong user.

6. Corrective Actions

1. Verify the correct service user and group.
2. Check directory and file permissions along the entire path.
3. Restore the appropriate ownership and permissions (e.g. chown and chmod with correct values).
4. Validate mounted volume permissions if using containers or cloud storage.
5. Check SELinux/AppArmor logs if applicable.
6. Restart the application after fixing the permissions.

7. Security Lesson

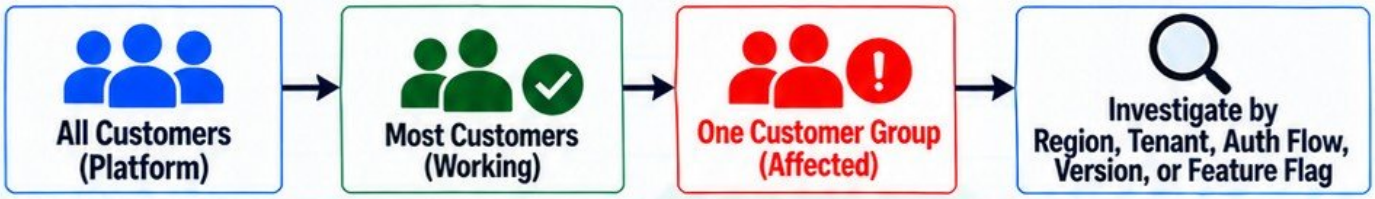
Never use chmod 777 as a universal permission fix. It grants excessive access, introduces security risks, and can hide the real problem. Always apply the minimum permissions required.

8. Key Judgment

Permission issues are often caused by deployment changes, incorrect ownership, or security controls. Fix the root cause with the correct permissions instead of applying overly permissive settings.

The Incident Looks Like an Outage, but Only One Customer Group Is Affected

Scenario: A monitoring alert reports failures, yet most customers can still use the platform.



1. Check Overall vs Segmented Error Rates

$ Example: error rate by customer group (monitoring dashboard or CLI)

Customer Group	Requests	Errors	Error Rate
All Customers	1,250,000	12,500	1.0%
Enterprise	600,000	3,000	0.5%
SMB	400,000	500	0.1%
Tenant: ACME	50,000	8,000	16.0%
Tenant: BETA	200,000	200	0.1%

Overall error rate looks low, but one customer group (Tenant: ACME) has a high failure rate.

2. Compare by Region, Tenant, or Authentication Flow

$ Example: filter logs by tenant or auth flow

```
$ grep "tenant_id=acme" /var/log/app.log | tail -5
2026-10-28 10:15:21  ERROR  tenant_id=acme
auth_service: token validation failed
2026-10-28 10:15:22  ERROR  tenant_id=acme
permission denied for resource
2026-10-28 10:15:23  WARN   tenant_id=acme
retrying request (attempt 3)
2026-10-28 10:15:24  ERROR  tenant_id=acme
upstream API returned 403
```

Logs show failures only for a specific tenant and authentication flow.

3. Check Application Version and Feature Flags

$ Example: compare versions and feature flags

Customer Group	App Version	Feature Flag X	Status
Enterprise	v1.4.2	OFF	Healthy
SMB	v1.4.2	OFF	Healthy
Tenant: ACME	v1.5.0	ON	Failing
Tenant: BETA	v1.4.2	OFF	Healthy

The affected customer group is on a different application version or has a specific feature flag enabled.

4. Use Request IDs to Trace the Failure Path

$ Example: trace a failed request using its ID

```
$ grep "req-7f9a3c2e" /var/log/app.log
2026-10-28 10:15:21  INFO   req-7f9a3c2e
tenant_id=acme   auth_service: validated token
2026-10-28 10:15:21  INFO   req-7f9a3c2e
calling feature-x service
2026-10-28 10:15:22  ERROR  req-7f9a3c2e
feature-x service returned 500
2026-10-28 10:15:22  ERROR  req-7f9a3c2e
request failed for tenant_id=acme
```

The request ID shows the exact dependency and component causing the failure.

5. Common Causes

- Issue isolated to one region or Availability Zone.
- Failures affecting only one tenant or customer segment.
- Authentication or authorization issues for a specific group.
- A new application version or feature flag enabled for one group.
- Data or configuration specific to one customer.
- Rate limits or quotas reached for a particular tenant.
- Downstream dependency failure affecting only a subset of traffic.

6. Investigation Steps

1. Confirm the scope: which customer group is affected.
2. Compare error rates by region, tenant, and endpoint.
3. Check recent deployments, feature flag changes, or configuration updates.
4. Use request IDs to trace the full request path.
5. Review authentication, authorization, and ownership settings.
6. Check downstream dependencies for group-specific failures.
7. Communicate findings and apply a targeted fix.

7. Key Judgment

Overall availability can hide severe impact on a smaller customer group.

8. Production Lesson

Always investigate incidents at the customer-segment level. A system can appear healthy overall while one customer group experiences significant failures.

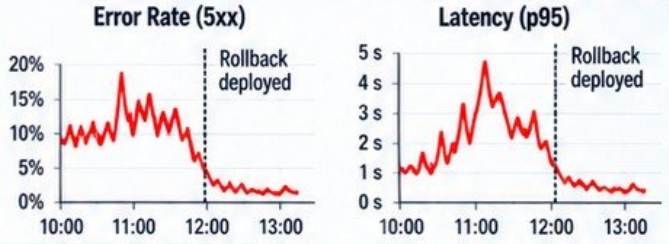
The First Fix Worked, but the Incident Is Not Over

Scenario: Error rates return to normal after rollback, but queued requests, failed background jobs, and delayed transactions remain.

Incident Detected (Errors Spike) → Rollback Deployed (Errors Return to Normal) → Queued Requests, Failed Jobs, Delayed Transactions Remain → Investigate and Clear Backlog → Full Service Recovery (Users Fully Restored)

1. Verify That Error Rates Have Actually Normalized

$ Check error rate and latency (example using Prometheus/Grafana or CLI)



Error rates and latency return to normal after the rollback, but this does not mean the incident is fully resolved.

2. Check Queue Depth and Background Jobs

$ Check message queue depth (example: AWS SQS)

Queue Name	Messages Visible	Messages In Flight	Age of Oldest
orders-queue	25,430	12,100	2h 15m
email-queue	3,200	800	45m
notification-queue	0	0	-

A large number of queued messages indicates background jobs are still behind and need time to process.

3. Check Background Worker Health and Throughput

$ Check worker status and processing rate

Worker	Status	Processing Rate	Queue Lag
order-worker	Running	5 msg/s	2h 10m
email-worker	Running	50 msg/s	12m
notification-worker	Running	120 msg/s	0m

Workers are running, but processing is slower than the backlog growth, so the queue is not clearing yet.

4. Verify Data Consistency and Customer Impact

$ Run application-specific checks

```
$ Check for failed or stuck transactions
$ Verify data consistency (e.g. missing orders, payments)
$ Review application logs for retry failures
$ Confirm customer-facing features work end-to-end
$ Check for manual intervention requirements
```

Ensure that queued work, failed transactions, and customer data are processed correctly and completely.

5. Common Remaining Issues

- Large backlog of queued requests or messages.
- Failed or stuck background jobs.
- Delayed transactions (e.g. payments, orders, emails).
- Data inconsistencies or partial updates.
- External system retries still pending.
- Rate limits or quotas still being hit after the rollback.
- Customers experiencing delays even though monitoring looks healthy.

6. Investigation Steps

1. Confirm error rates and latency have normalized.
2. Check queue depth and message age.
3. Verify background worker health and processing rate.
4. Review failed jobs and retry mechanisms.
5. Validate data consistency and completion of transactions.
6. Test customer-facing functionality end-to-end.
7. Monitor systems until all backlogs are cleared.
8. Communicate status and estimated recovery time.

7. Key Judgment

Restoring a healthy deployment is not the same as completing service recovery. The incident continues until all backlogs, failed jobs, and customer impacts are fully resolved.

8. Production Lesson

Always verify system recovery beyond error rates. Check queues, background jobs, data consistency, and real customer impact before declaring the incident closed.

The 30-Minute Incident Challenge: Investigate Before You Change Anything

Final Scenario: An e-commerce application running behind AWS ALB, Nginx, Kubernetes, and PostgreSQL begins returning intermittent 502 and 504 errors immediately after a deployment.

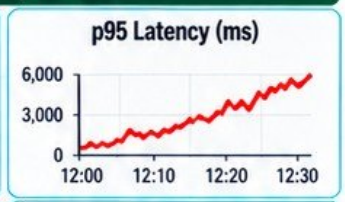


1. Evidence Observed After Deployment

- Increased p95 latency and HTTP 5xx errors.
- Kubernetes Pods reporting mixed readiness.
- Nginx upstream timeout logs.
- Database connection pool utilization near its limit.
- A recent configuration change.

2. Key Metrics and Symptoms





Kubernetes Pod Readiness

POD NAME	READY	STATUS
web-7d8f9	1/1	Running
api-6c9d8	0/1	NotReady
api-6c9d9	1/1	Running

Database Connection Pool



92% Used

92 / 100 connections

3. Construct a Timeline

Time (UTC)	Event	Details
12:00	Deployment	New version deployed to Kubernetes
12:02	Errors increase	5xx errors and latency rising
12:05	Nginx timeouts	Upstream timeout errors in logs
12:10	Mixed readiness	Some pods NotReady
12:15	DB pool high	Connection pool at 92%
12:20	Customer reports	Users experiencing intermittent 502/504

4. Competing Hypotheses

1. New deployment introduced a breaking change.
2. Application pods are unhealthy or overloaded.
3. Nginx upstream timeout due to slow application responses.
4. Database connection pool exhaustion causing request timeouts.
5. Misconfiguration (environment variables, resource limits, or feature flag).
6. External dependency or network issue.

5. Safe Diagnostic Commands (Do NOT Change Anything Yet)

```
# Check ALB target health
aws elbv2 describe-target-health --target-group-arn <tg-arn>
# Check Kubernetes pod status
kubectl get pods -o wide
kubectl describe pod <pod-name>
# Check Nginx upstream errors
tail -n 50 /var/log/nginx/error.log
# Check database connection usage
SELECT count(*) FROM pg_stat_activity;
# Check recent changes
kubectl rollout history deployment/<name>
```

6. Recommended Containment Actions

1. If a new version is confirmed as the cause, prepare a rollback plan.
2. Consider scaling pods if they are healthy but underprovisioned.
3. Increase database connection pool limit temporarily (if safe) while investigating root cause.
4. Throttle or disable non-critical background jobs if they are consuming connections.
5. Communicate with stakeholders and provide regular updates.

Final Deliverable: 30-Minute Investigation Record

Timestamp	Observed Fact	Hypothesis	Action Taken	Result	Next Step
12:00	Deployment completed	Config change issue	Checked rollout history	New config found	Review and test rollback
12:05	5xx errors increased	Pod or app issue	Checked pod status	Mixed readiness	Investigate unhealthy pods
12:10	Nginx upstream timeouts	Slow app or DB	Checked Nginx logs	Upstream timeout	Check app logs and DB
12:15	DB pool at 92%	DB pool exhaustion	Checked DB activity	High connections	Identify long-running queries
...	...	...	...	...	...

8. Key Judgment

Restoring a healthy deployment is not the same as completing service recovery. You must verify that all backlogs are cleared, errors are resolved, and customer-facing functionality is fully restored.